INFORMATION TECHNOLOGY UNIVERSITY

Department of Computer & Software Engineering – ITU

SE411L / MD442L: Mobile App Development

Course Instructor: Usama Bin Shakeel	**Dated:** 19th Sep 2026
Teaching Assistant: Usman,Raza,Afaq	**Semester:** Fall 2026
Session: 2026–2030	**Batch:** 2023

Lab 3. Quran Index

Name	Roll Number	Obtained Marks / 35

Checked on: _____

Signature: _____

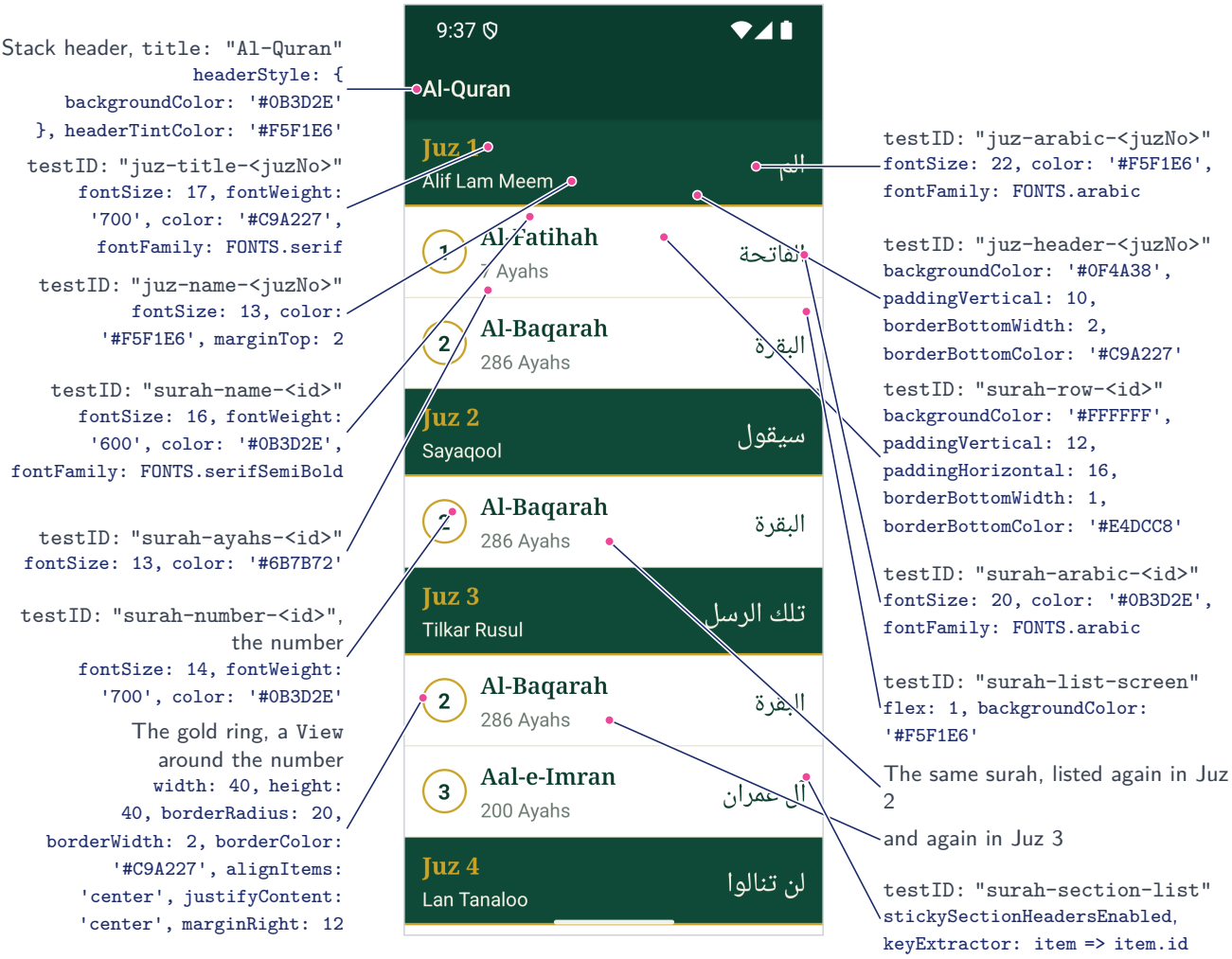
Equipment and Components

Component / Tool	Description	Quantity
Computer with Node LTS	Available in lab	1
Physical device or Android emulator	Android Studio / Xcode	1
React Native CLI + Metro	Installed via npm	—

Objective

Build a two-screen Quran index from one bundled JSON file: a `SectionList` of the 30 juz, each listing the surahs that fall in it, and a detail screen a surah opens onto. Names like `surah-row-1-2` on the screenshots are `testID`s.

Screen 1: Surah Index



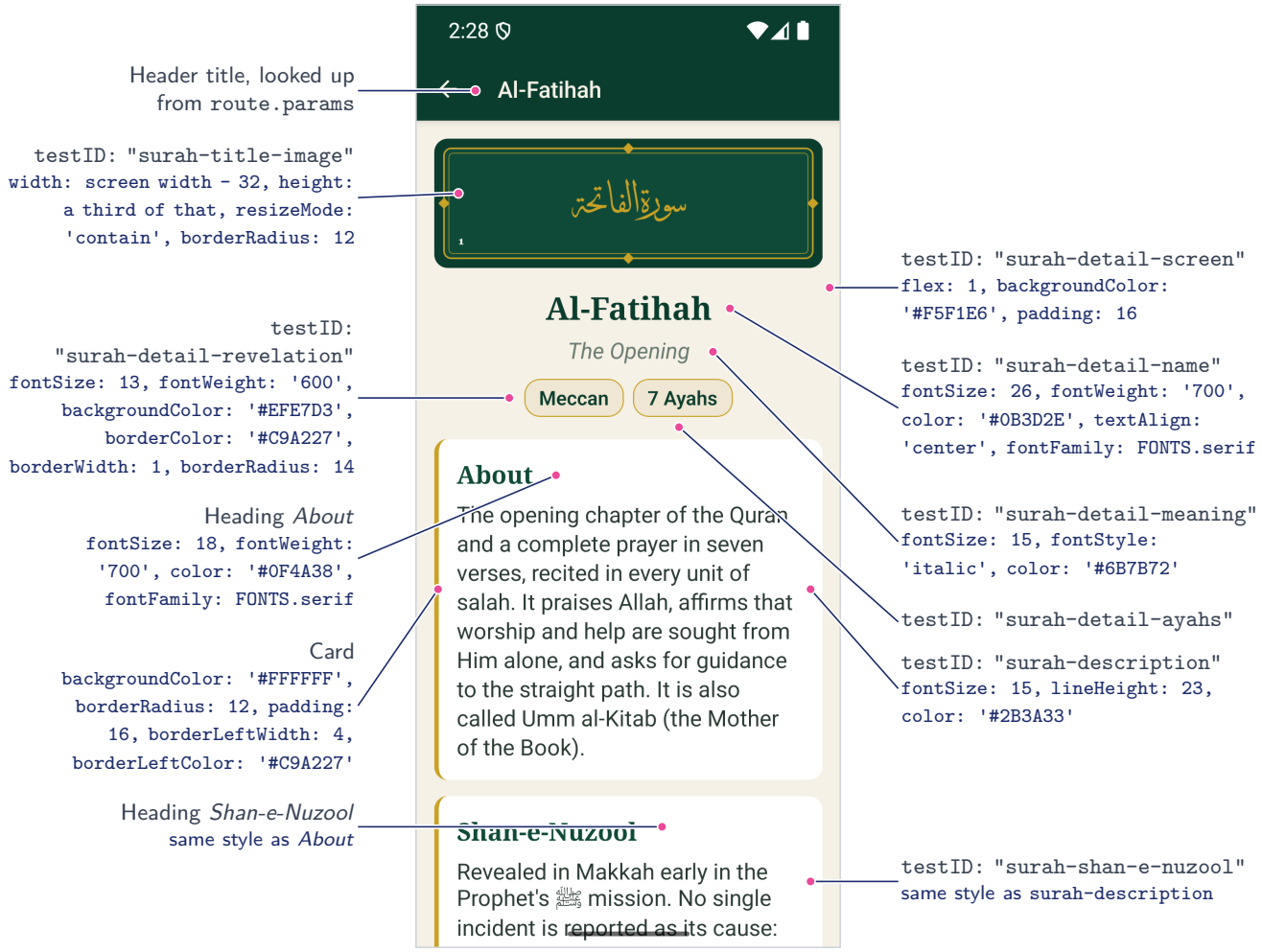
On screen: Thirty juz headers, each followed by the surahs that fall in it and each surah's ayah count.

Assets: The three fonts in `FONTS`, from `src/theme/fonts.js`. Every word on this screen comes from `src/data/quran.json`.

Parent style: `flex: 1, backgroundColor: '#F5F1E6'`.

Does: Headers stick while their juz scrolls; pressing a surah opens its detail screen.

Screen 2: Surah Detail



On screen: The surah's title banner, its name and meaning, two chips, and the *About* and *Shan-e-Nuzool* cards.

Assets: The banner file each surah names in `titleImage`, from `src/assets/titles/`, reached through `titleImages.js`.

Parent style: `flex: 1, backgroundColor: '#F5F1E6', padding: 16`.

Does: Reads `route.params.surahNo` and looks every field up from the JSON. The screen scrolls.

Task Description

Build it in this order.

1. **The app**: one native stack, `SurahList` then `SurahDetail`, opening on `SurahList` titled `Al-Quran`. `App.js` is written for you in the starter.

2. **The data**: everything on both screens is read from `src/data/quran.json`, which holds 30 juz, 114 surahs and 6,236 ayahs.
   - `import` the JSON. Do not fetch it, and do not edit it.
   - It has two tables: each juz lists only the ids of its surahs, `"surahs": [1, 2]`, and each surah's details are in `surahs`, once.
   - No surah name, juz name or ayah number is typed into a component.
3. **`quranData.js`** exports three functions: `buildSections()`: one section per juz, titled `Juz 1` to `Juz 30`, each surah id in the juz looked up in the `surahs` table for its name, Arabic name and ayah count. A row `id` is the juz and the surah together, `1-2`, and is unique across the whole list. `findSurah(n)` looks a surah up by its `id` and returns nothing for a number that is not one. `formatAyahCount(n)` writes `286 Ayahs`.
4. **The list screen**: a `SectionList`, not one flattened `FlatList`.
   - `keyExtractor` returns the row `id`, never the index, and headers come from `renderSectionHeader`.
   - A surah listed by several juz gets a full row in each.
5. **The detail screen**, opened by pressing a row: Only the surah number travels in the params; every other field is looked up from the JSON. The title image is the file `quran.json` names in the surah's `titleImage`, a bundled `require` sized from the window at 3:1 and contained, never stretched.
6. **Fonts**: `src/theme/fonts.js` exports `FONTS`, and the font files are already in the starter. Set `fontFamily` from it, never as a typed name: `FONTS.serif` for the juz titles, the surah name on the detail screen and the *About* and *Shan-e-Nuzool* headings; with `fontWeight: '700'` Android uses the bold file. `FONTS.serifSemiBold` for the surah names in the list, with `fontWeight: '600'`. `FONTS.arabic` for every Arabic name. Everything else (juz names, surah numbers, ayah counts, the meaning, the chips and the card text) has no `fontFamily` and uses the system font.
7. **Files**: the starter has every file. File *names* are fixed and folders are free, so tidy `src/` however you like, but keep these names and the `.js` extension: `App.js`, `SurahListScreen`, `SurahDetailScreen`, `JuzHeader`, `SurahRow`, `quranData`, `quran.json`.
8. **testIDs**: use every name labelled on the screenshots above, spelled exactly, on the element it points to.
   - Row and header names end in the row `id`, never the index.
9. **Rules**: no class components, no remote data, no extra list or navigation libraries, and write each image path as a plain `require('...')` string. `SurahRow` and `JuzHeader` hold no state of their own.
10. **Run it** with `npx react-native run-android`, then `npm test`. The shared tests start red and pass as you finish; hidden tests are also part of your mark.

References

Read the JSON guide, `COMPONENTS.md`, before you start: keys and values, reading and searching a bundled JSON file, and holding it in React Native state. `SectionList` builds on the `FlatList` from Lab 02, with the same `renderItem` and `keyExtractor` plus `sections` and `renderSectionHeader`. `App.js` is written for you, so the stack navigator is a worked example to read rather than something to build.

INFORMATION TECHNOLOGY UNIVERSITY

Assessment Rubric for Lab

Performance metric	CLO	Able to complete the task over 80% (4–5)	Able to complete the task 50–80% (2–3)	Able to complete the task below 50% (0–1)	Marks
Realization of experiment	1	Executes without errors excellent user prompts, good use of symbols, spacing in output. The testing has been completed.	Executes without errors, user prompts are understandable, minimum use of symbols or spacing in output. Some testing has been completed.	Does not execute due to syntax errors, runtime errors, user prompts are misleading or non-existent. No testing has been completed.	
Conducting experiment	1	Able to make changes and answer all questions.	Partially able to make changes and few incorrect answers.	Unable to make changes and answer all questions.	
Computer use	2	Document submission timely.	Document submission late.	Document submission not done.	
Teamwork	3	Actively engages and cooperates with other group member(s) in an effective manner.	Cooperates with other group member(s) in a reasonable manner but conduct can be improved.	Distracts or discourages other group members from conducting the experiment.	
Laboratory safety and disciplinary rules	3	Code comments are added and do help the reader to understand the code.	Code comments are added and do not help the reader to understand the code.	Code comments are not added.	
Data collection	3	Excellent use of white space, creatively organized work, excellent use of variables and constants, correct identifiers for constants, No line-wrap.	Includes name, and assignment, white space makes the program fairly easy to read. Title, organized work, good use of variables.	Poor use of white space (indentation, blank lines) making code hard to read, disorganized and messy.	
Data analysis	4	Solution is efficient, easy to understand, and maintain.	A logical solution that is easy to follow but it is not the most efficient.	A difficult and inefficient solution.	
				Total (out of 35):	